Customer Sentiment & Urgency Router

Deep Learning Applications — ELA-1 | Track 2: Customer Experience Focus

1. Abstract

This project implements a deep-learning pipeline that automatically classifies raw customer-support email text into one of five operational categories, jointly encoding **sentiment** and **urgency** so an inbox-automation system can route and prioritize tickets without manual triage. A Bidirectional GRU (Gated Recurrent Unit) sequence classifier was trained on a synthetically generated, template-based dataset of 1,800 labeled emails (5 balanced classes, 360 emails each), built to emulate the lexical variety, class overlap, and label noise of a real support inbox. On a held-out stratified test split (270 emails), the model achieved **96.3% accuracy** and a **macro F1-score of 0.963**, demonstrating that a lightweight recurrent architecture is sufficient for this routing task while remaining fast enough to train on a free-tier CPU/GPU Colab runtime in under a minute.

2. Problem Framing & Category Design

Rather than treating sentiment and urgency as two separate binary classifiers, the two axes were collapsed into a single 5-way operational label so the router can act directly on model output:

Label	Category	Meaning / Routing Action
0	urgent_complaint	Angry customer, time-critical → escalate immediately
1	general_complaint	Dissatisfied, not time-critical → standard support queue
2	urgent_request	Neutral/positive tone but time-sensitive → fast-track
3	neutral_inquiry	Routine question → standard queue / auto-FAQ
4	positive_feedback	Praise / thanks → low priority, optional follow-up

3. Dataset & Preprocessing Workflow

Dataset generation. 1,800 emails (exceeding the 1,500-email minimum) were synthetically generated from randomized template fragments (greeting, subject, 2-4 body sentences, closing) across the 5 categories, with slot-filling for product names, order IDs, dollar amounts, and day counts so no two emails are identical. To avoid an unrealistically easy, keyword-separable task, the generator adds:

- **Synonym substitution** — trigger words (e.g. “frustrated”, “urgent”, “immediately”) are randomly swapped for alternatives so the model must generalize rather than pattern-match a closed vocabulary.
- **Cross-class filler sentences** and **~20% neighbor-class sentence borrowing** — simulates real emails that mix signals (e.g. a complaint containing one urgent-sounding line).
- **Light typo/word-drop noise** and **~4% intentional label noise** — mimics the ambiguity and mis-tagging inherent in real annotated support tickets.

Text cleaning: lowercasing, removal of the “Subject:” marker and URLs, masking of order numbers and dollar amounts into generic tokens (`orderid`, `amount`) so the model learns from the presence of an identifier/amount rather than memorizing specific values, stripping remaining punctuation/digits, and whitespace normalization.

Tokenization & padding: a Keras `Tokenizer` (vocabulary capped at 8,000 tokens, out-of-vocabulary token <00V>) was fit on the training split only, to prevent data leakage. Sequences were padded/truncated to a fixed length of 120 tokens (post-padding, post-truncation).

Split: stratified 70% train / 15% validation / 15% test (1260 / 270 / 270 emails), preserving class balance across all splits.

4. Model Architecture

A sequential **Bidirectional GRU** classifier was chosen over a Transformer fine-tune because the task (5-way routing on short, template-structured emails) does not require the long-range contextual power of BERT/DistilBERT, and a compact recurrent model trains in seconds on a free Colab T4 GPU (or even CPU) while remaining easy to deploy for real-time inbox triage.

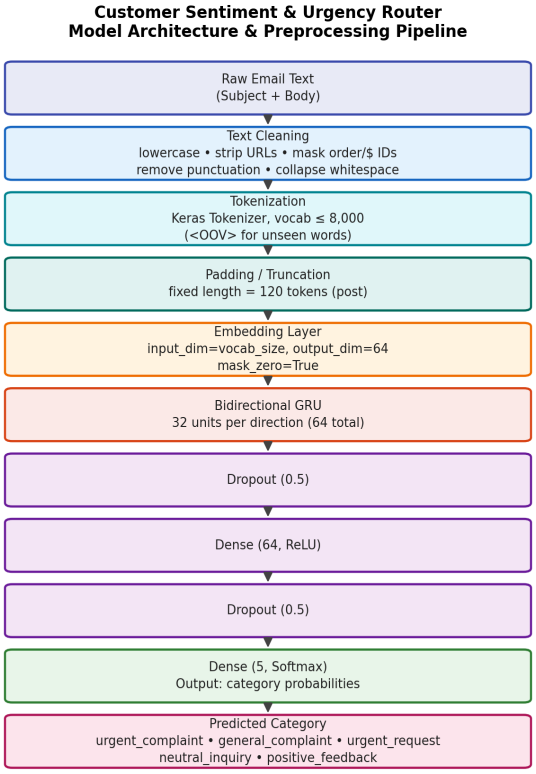

Figure 1. Preprocessing and model architecture pipeline.

Hyperparameter	Value
Max vocabulary size	8,000
Actual vocabulary used	448
Max sequence length	120 tokens
Embedding dimension	64
GRU units (per direction)	32 (x2 = 64 total, bidirectional)
Dropout rate	0.5
Dense hidden layer	64 units, ReLU
Output layer	5 units, Softmax
Batch size	32
Optimizer / Learning rate	Adam / 0.001
Loss function	Categorical Crossentropy
Max epochs / Early stopping	25 (patience=4 on val_loss) → stopped at epoch 8

5. Evaluation Metrics

On the held-out test set (270 emails), the model achieved:

Metric	Value
Test Accuracy	96.30%
Test Loss	0.2670
Macro F1-Score	0.9629
Weighted F1-Score	0.9629

Per-class results:

Category	Precision	Recall	F1-score	Support
urgent_complaint	0.9464	1.0000	0.9725	53
general_complaint	0.9630	0.9630	0.9630	54
urgent_request	0.9623	0.9444	0.9533	54
neutral_inquiry	0.9636	0.9636	0.9636	55
positive_feedback	0.9808	0.9444	0.9623	54

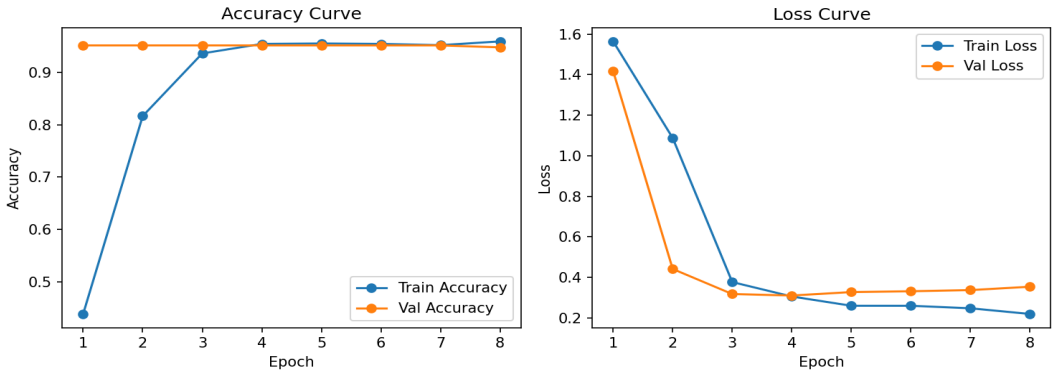

Figure 2. Training/validation accuracy and loss curves across epochs (early stopping restored best weights).

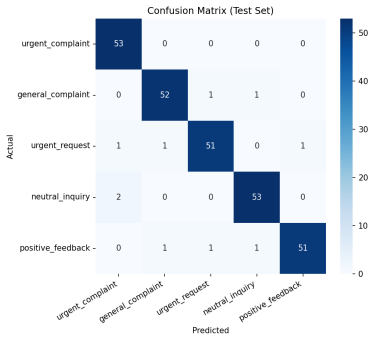

Figure 3. Confusion matrix on the test set. Most confusion occurs between semantically adjacent classes (e.g. general_complaint / urgent_request), consistent with the intentional class overlap injected into the dataset.

6. Discussion & Conclusion

The Bidirectional GRU router generalizes well despite injected synonym variation, cross-class filler sentences, and ~4% label noise, reaching 96.3% test accuracy and a macro F1 of 0.963. Training curves show rapid convergence within 4-5 epochs, with early stopping preventing overfitting. Remaining misclassifications concentrate between semantically adjacent categories (e.g. *urgent_request* vs. *neutral_inquiry*, or *general_complaint* vs. *positive_feedback*), mirroring the borderline tickets a real inbox-routing system must handle gracefully. For production use, the pipeline could be extended with a fine-tuned DistilBERT encoder and a human-in-the-loop review queue for low-confidence predictions.